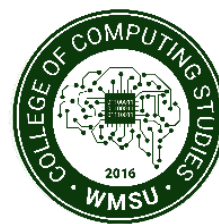




Republic of the Philippines
Western Mindanao State University
College of Computing Studies
DEPARTMENT OF INFORMATION TECHNOLOGY
Zamboanga City



**VisionPool: A Real-Time Pool Rule Compliance Monitoring System
Using Object Detection and Automated Flagging for Administrative Oversight**

A CAPSTONE PROJECT

Presented to the Faculty of the

College of Computing Studies
Western Mindanao State University

In Partial Fulfillment
of the Requirements for the Degree
Bachelor of Science in Information Technology

SOLIS, LUISA MAE C.
Researchers

Jason A. Catadman and Jhon Paul I. Arip

June, 2027

TABLE OF CONTENTS

TITLE	i
-------------	---

Chapter	Page
---------	------

INTRODUCTION.....	1
1.1 Project Context.....	1
1.2 Statement of the Problem	2
1.3 Purpose and Description	3
1.4 Objectives.....	3
1.4.1 Specific Objectives	4
1.5 Scope and Limitation.....	4
1.6 Significance of Study.....	5
1.7 Definition of Terms	6
2 REVIEW OF RELATED LITERATURE AND SYSTEMS	12
2.1 Introduction.....	12
2.1.1 Development of YOLOv3 for Multi-Class Object Detection.....	12
2.1.2 YOLOv4 — Optimal Speed and Accuracy of Object Detection.....	13
2.1.3 YOLOv8 — A New Frontier in Real-Time Object Detection.....	13
2.1.4 You Only Look Once — Unified Real-Time Object Detection	13
2.1.5 Automated PPE Detection Using YOLOv3 in Construction SitesRRL 4.....	14
2.1.6 Face Mask Compliance Detection Using YOLOv5 During COVID-19.....	14
2.1.7 Automated Social Distancing Monitoring Using Computer Vision	15
2.1.8 Swimmer Behavior Recognition and Drowning Detection in Indoor Swimming Pools	15
2.1.9 Intelligent Traffic Monitoring System Based on YOLO and Convolutional Fuzzy Neural Networks.....	16
2.1.10 Deep Learning for Site Safety — Real-Time Detection of Personal Protective Equipment	16
2.1.11 Two-Stream Convolutional Networks for Human Action Recognition in Videos	17
2.1.12 Pose Estimation for Human Activity Detection in Sports Environments	17
2.1.13 Fall Detection for Elderly Care Using YOLOv5 and OpenPose Integration.....	17

2.1.14	Abnormal Crowd Behavior Detection Using Motion Information Images and Convolutional Neural Networks	18
2.1.15	Large-Scale Clothing Recognition and Attribute Prediction Using Deep Learning	18
2.1.16	Automated PPE Compliance Monitoring Using YOLOv8 for Construction Safety	19
2.1.17	Face Mask Detection Using YOLOv2 and ResNet-50 for Public Health Compliance	19
2.1.18	Real-Time Face Mask and Social Distancing Violation Detection System Using YOLO	20
2.1.19	Automatic Real-Time Detection of Drowning Using YOLOv5 and Faster R-CNN for Video Surveillance	20
2.1.20	Monitoring COVID-19 Social Distancing Using Person Detection and Tracking	21
2.2	Summary	24
2.3	Synthesis	24
3	TECHNICAL BACKGROUND	26
3.1	Technology Requirements	26
4	DESIGN AND METHODOLOGY	32
4.1	Research Design	32
4.2	Developmental Methodology.....	32
4.3	System Architecture	33
4.4	Data Collection Methods	33
4.4.1	Data Source.....	33
4.4.2	Data Gathering Instrument.....	34
4.4.3	Data Gathering Technique and Procedures	34
4.5	Data Analysis.....	35
4.5.1	Context Diagram	35
4.5.2	Data Flow Diagram.....	36
4.5.3	Flowchart	38
4.5.4	Use Case	39
4.6	Requirement Specification	41
4.6.1	Functional Requirements	42
4.6.2	Non-Functional Requirements	44
4.7	System Design	46
4.7.1	Database Design.....	47
4.7.2	Entity Relationship Diagram.....	47
4.7.3	Database Fields.....	48

4.8	Tools and Technology	49
4.9	Evaluation and Testing.....	50
REFERENCES		52

LIST OF TABLES

Table 1. Definition Of Terms	11
Table 2. Review of Related Works and System Comparison Table	22
Table 3. Use Case 1.....	40
Table 4. Use Case 2.....	40
Table 5. Use Case 3.....	41
Table 6. Use Case 4.....	41
Table 7. User Characteristics.....	44
Table 8. TBLViolations	48
Table 9. TBLUsers	49

LIST OF FIGURES

Figure 1. Conceptual Framework	26
Figure 2. Context Diagram.....	35
Figure 3. Data Flow Diagram.....	36
Figure 4. Flowchart	38
Figure 5. Use Case	39
Figure 6. Database Design	47
Figure 7. Entity Relationship Diagram.....	47

CHAPTER 1

INTRODUCTION

1.1 Project Context

Public swimming pools are among the most widely patronized recreational facilities, offering communities access to leisure, fitness, and competitive sports activities. In the Philippines, public and private pool facilities are governed by a combination of facility-specific rules and public health standards that aim to protect patrons and maintain operational order. Common pool policies include the mandatory use of entry wristbands to verify paid admission, requirements for proper swimming attire, and designated restrictions against jumping or diving in non-sanctioned areas. These rules are essential not only for ensuring fair access to the facility but also for preventing accidents and injuries.

Despite the importance of consistent rule enforcement, most pool facilities continue to rely on manual monitoring by lifeguards and pool attendants. This approach is inherently limited by human attention span, staffing levels, and the physical constraints of monitoring expansive pool areas during peak hours when patron density is highest. Violations frequently go unnoticed or are addressed inconsistently, leading to potential safety hazards and disputes between patrons and staff.

Recent advances in computer vision and deep learning have demonstrated significant potential for automating visual monitoring tasks in public spaces. YOLOv8 (You Only Look Once, version 8), developed by Ultralytics, is one of the most efficient and accurate real-time object detection frameworks available today. Its ability to process live video streams with minimal computational overhead makes it well-suited for deployment in resource-constrained environments such as local pool facilities. Paired with affordable IP cameras and web-based administrative tools, YOLOv8 enables the construction of cost-effective automated monitoring systems that can operate continuously without fatigue.

VisionPool is proposed as a direct response to the operational limitations of manual pool rule enforcement. By deploying a YOLOv8-based detection pipeline connected to a Tapo IP camera and a Django-powered administrative dashboard, VisionPool aims to provide pool management staff with a reliable, data-driven tool for real-time rule compliance monitoring, automated violation flagging, and incident documentation.

1.2 Statement of the Problem

Public swimming pool facilities face persistent challenges in the consistent and reliable enforcement of safety and operational rules. Specifically, the following problems have been identified:

1. Manual monitoring of pool rules is labor-intensive and subject to human error. Lifeguards and pool attendants cannot maintain constant vigilance over all patrons simultaneously, particularly during peak hours, resulting in undetected or inconsistently addressed violations.
2. The absence of a systematic violation documentation process makes it difficult for pool management to track compliance trends, identify repeat violators, or provide evidence-based justification for operational decisions or disciplinary actions.
3. Existing pool management systems do not integrate real-time visual monitoring capabilities, leaving a significant gap between administrative oversight tools and actual on-ground rule enforcement.
4. Manual rule enforcement places additional stress on pool staff and may lead to confrontational situations with patrons when violations are raised inconsistently.

These problems collectively reduce the effectiveness of pool safety protocols and increase the risk of patron injury, unauthorized access, and non-compliance with facility regulations. VisionPool addresses these issues by automating the detection and documentation of rule violations through computer vision technology.

1.3 Purpose and Description

VisionPool is a real-time pool rule compliance monitoring system designed to automate the identification and documentation of rule violations in public swimming pool facilities. The system uses a YOLOv8-based object detection model trained on pool environment images to detect three primary violation types from live IP camera feeds: the absence of entry wristbands, non-compliance with proper swimming attire requirements, and prohibited jumping or diving activities.

Upon detecting a violation, the system automatically captures a screenshot of the flagged incident, appends a timestamp and violation type label, and transmits the record to a Django-based web administrative dashboard accessible to pool management staff. The dashboard provides a real-time display of flagged incidents and maintains a searchable violation log for administrative review, reporting, and record-keeping purposes.

The system is designed for deployment in a local network environment using an affordable Tapo IP camera as the video input source and OpenCV as the video processing framework. VisionPool is intended to complement existing pool management operations rather than replace human staff, providing an additional layer of automated oversight that improves enforcement consistency and reduces the manual workload of pool attendants and administrators.

1.4 Objectives

General Objective: To design, develop, and evaluate a real-time pool rule compliance monitoring system that uses object detection to automatically flag violations and relay them to an administrative dashboard for oversight and documentation.

1.4.1 Specific Objectives

1. To develop a YOLOv8-based detection model capable of identifying wristbands, swimming attire compliance, and jumping or diving activities from live IP camera feeds.
2. To implement an automated flagging and screenshot capture mechanism that records violation incidents with timestamps and violation type labels.
3. To develop a web-based administrative dashboard using Django that displays flagged incidents in real time and maintains a searchable and filterable violation log.
4. To integrate the detection pipeline with a Tapo IP camera input via OpenCV for real-time video stream processing.
5. To evaluate the accuracy, precision, recall, and mean Average Precision (mAP) of the detection model on a locally collected and annotated dataset of pool environment images.

1.5 Scope and Limitation

VisionPool covers the following functionalities and operational boundaries:

The system detects three specific rule violation types: (1) absence of entry wristbands, (2) improper swimming attire, and (3) prohibited jumping or diving activities. Detection is performed in real time from a single Tapo IP camera feed processed via OpenCV.

The administrative dashboard, developed using the Django web framework, provides real-time display of flagged violations, a searchable violation log, and screenshot viewing capabilities. The system is intended for deployment on a local network accessible to authorized pool management staff.

The YOLOv8 model is trained and evaluated on a locally collected dataset of pool environment images. The system operates on a desktop or laptop computer connected to the same local network as the camera.

The following limitations apply to the current scope of the project: The system is designed for single-camera deployment and does not natively support multi-camera configurations in this version. Detection accuracy is dependent on camera placement, image resolution, and lighting conditions. The system does not perform facial recognition or patron identification. The violation detection model may require periodic retraining as pool environment conditions or rule definitions change. The system does not automatically trigger alerts to external agencies and is intended solely for internal administrative use.

1.6 Significance of Study

VisionPool addresses a practical and underserved need in the management of public swimming pool facilities. Its significance extends to the following stakeholders:

Pool Management Staff and Administrators: The primary beneficiaries of VisionPool are pool management personnel, including pool managers, supervisors, and administrative staff. The system reduces the manual burden of rule enforcement monitoring, provides a reliable and objective record of violations, and enables data-driven decision-making regarding pool operations, staffing, and patron compliance.

Lifeguards and Pool Attendants: By providing an automated violation detection layer, VisionPool allows lifeguards and pool attendants to focus their attention on patron safety and emergency response rather than constant rule monitoring. This reduces occupational stress and the risk of confrontational interactions with patrons over rule enforcement.

Pool Patrons: A consistently enforced rule environment benefits all pool patrons by ensuring fair application of facility regulations, reducing the risk of accidents caused by rule violations such as unauthorized jumping or improper attire, and maintaining the overall quality of the recreational experience.

The Field of Computer Vision and Public Safety: VisionPool contributes to the growing body of applied research on computer vision systems in public facility management. The project demonstrates the feasibility of deploying affordable, real-time

object detection systems in resource-constrained environments and provides a replicable framework for similar monitoring applications in recreational and public safety contexts.

Western Mindanao State University: As a capstone research output, VisionPool contributes to the institutional knowledge base of the Department of Information Technology and demonstrates the practical application of advanced machine learning techniques in addressing real-world community needs.

1.7 Definition of Terms

The following terms are defined as used in this study:

Term	Definition
1. Administrative Dashboard	A web-based graphical user interface that consolidates critical data, metrics, and system status information onto a single display, enabling authorized administrators to monitor, manage, and oversee system operations in real time. In the context of this study, it refers to the Django-based interface used by pool management staff to view flagged violations, access incident screenshots, and review the violation log. ^[1]
2. Automated Flagging	The process by which a software system automatically identifies, marks, and records a detected rule violation or anomalous event for administrative review without requiring manual intervention. As used in this study, automated flagging refers to the VisionPool pipeline's ability to detect pool rule violations via YOLOv8 and immediately log them to the administrative dashboard. ^[2]
3. Bounding Box	A rectangular region defined by coordinate values that is drawn around a detected object within an image or video

Term	Definition
	frame to indicate its spatial location and extent. In object detection, bounding boxes are the primary output representation used to localize detected objects alongside their class labels and confidence scores. ^[3]
4. Deep Learning	A branch of machine learning that uses artificial neural networks with multiple processing layers to learn hierarchical representations of data, enabling computers to recognize complex patterns in images, audio, and text. Deep learning forms the foundational computational approach underlying the YOLOv8 object detection model used in this study. ^[4]
5. Django	An open-source, high-level web application framework written in Python that follows the Model-View-Template (MVT) architectural pattern and emphasizes rapid development, clean design, and built-in security features. In this study, Django serves as the backend framework for the VisionPool administrative dashboard, handling database operations, user authentication, and API endpoints. ^[5]
6. Fine-tuning	A transfer learning technique in which a pre-trained machine learning model is further trained on a smaller, domain-specific dataset to adapt its learned representations to a new but related task. In this study, fine-tuning refers to the process of training the pre-trained YOLOv8 model on a locally collected pool environment dataset to enable it to detect pool-specific rule violations. ^{[4] [6]}

Term	Definition
7. IP Camera	An Internet Protocol camera that captures video footage and transmits the data as a digital signal over a wired or wireless network, enabling remote access to live video streams. In this study, a Tapo IP camera is used as the primary video input device for monitoring the pool area via the RTSP protocol. ^[7]
8. Mean Average Precision (mAP)	A standard quantitative evaluation metric for object detection models, computed as the mean of the Average Precision (AP) values calculated across all object detection classes at a specified Intersection over Union (IoU) threshold. mAP provides a single comprehensive score that balances both detection precision and recall across all violation classes in the VisionPool model. ^[8]
9. Object Detection	A computer vision task that simultaneously identifies the presence, category, and spatial location of one or more objects within an image or video frame by combining object classification with object localization, typically represented through bounding boxes and class labels. Object detection is the core computational task performed by the YOLOv8 model in the VisionPool system. ^{[3] [8]}
10. OpenCV	Open Source Computer Vision Library; a free, open-source library of programming functions primarily designed for real-time computer vision applications, supporting operations including image processing, video capture, object detection, and feature extraction. In this study, OpenCV is used to capture and process live video streams from the Tapo IP camera via the RTSP protocol. ^[9]

Term	Definition
11. Precision	An evaluation metric in machine learning and object detection defined as the ratio of true positive detections to the total number of positive detections made by the model (true positives plus false positives). High precision indicates that when the VisionPool system flags a violation, it is likely to be a genuine violation rather than a false alarm. ^[8]
12. Real-Time Processing	A mode of data processing in which a system receives, analyzes, and responds to input data immediately as it is generated, with response times sufficiently low to support time-sensitive decision-making and monitoring tasks. In this study, real-time processing refers to the VisionPool pipeline's continuous frame-by-frame analysis of the live IP camera video stream. ^[10]
13. Recall	An evaluation metric in machine learning and object detection defined as the ratio of true positive detections to the total number of actual positive instances in the dataset (true positives plus false negatives). High recall in the VisionPool model indicates that the system successfully detects most actual rule violations occurring in the camera feed. ^[8]
14. Roboflow	A cloud-based computer vision platform that provides tools for uploading, annotating, augmenting, preprocessing, and managing image datasets for machine learning model training, with support for exporting datasets in multiple detection framework formats including YOLOv8. In this study, Roboflow is used for bounding box annotation of the pool environment

Term	Definition
	training dataset and for dataset export in YOLOv8 format. [11]
15. RTSP (Real-Time Streaming Protocol)	A network control protocol designed for use in entertainment and communications systems to control the streaming of media servers, enabling the establishment and control of media sessions between endpoints over IP networks. In this study, RTSP is the protocol used by OpenCV to access the live video feed from the Tapo IP camera over the local network. [12]
16. Screenshot Capture	The automated process of saving a still image extracted from a live video stream at the specific moment a rule violation is detected, used to create a timestamped visual record of each flagged incident for administrative review and documentation. In this study, screenshot capture is performed by the VisionPool detection pipeline using OpenCV upon confirmed violation detection by the YOLOv8 model. [2]
17. Wristband	A band or bracelet worn around the wrist, used in recreational facility management as a physical token to indicate that a patron has paid for entry or meets access requirements. In this study, the presence or absence of an entry wristband on pool patrons is one of the three primary rule compliance indicators monitored by the VisionPool detection model.
18. YOLOv8	You Only Look Once version 8; a state-of-the-art, single-stage, anchor-free real-time object detection framework developed by Ultralytics that processes an entire image in a single forward pass through a convolutional neural

Term	Definition
	network, achieving high detection accuracy at real-time inference speeds. YOLOv8 serves as the core machine learning model of the VisionPool detection pipeline. ^[13]

Table 1. Definition Of Terms

CHAPTER 2

REVIEW OF RELATED LITERATURE AND SYSTEMS

2.1 Introduction

The development of VisionPool draws upon an expanding body of research spanning real-time object detection, computer vision-based compliance monitoring, human activity recognition, wearable and clothing attribute detection, automated surveillance systems, and web-based administrative interfaces for institutional monitoring. The following review examines twenty related works published between 2016 and 2026 that share thematic and technical relevance with the proposed study. The reviewed works are organized into five thematic clusters: (1) YOLO-based object detection frameworks; (2) computer vision for compliance and safety monitoring; (3) human activity and behavior recognition; (4) clothing and accessory attribute detection; and (5) web-based monitoring dashboards and administrative systems. Each entry discusses the study's contribution, methodology, key findings, and its relevance and limitations relative to VisionPool.

The following literature and systems exhibit comparable structures and thematic relevance.

YOLO-Based Object Detection Frameworks

2.1.1 Development of YOLOv3 for Multi-Class Object Detection

Redmon and Farhadi (2018) ^[14] presented YOLOv3, an improved iteration of the YOLO object detection framework that introduced multi-scale detection using feature pyramid networks, enabling more accurate detection of objects at varying sizes within a single image. The study demonstrated state-of-the-art detection speed and competitive accuracy on the COCO benchmark dataset, establishing YOLOv3 as one of the most widely adopted frameworks for real-time detection tasks. This work is foundational to VisionPool in that it established the multi-scale feature extraction principles that were refined in subsequent YOLO versions, including YOLOv8. The limitation of YOLOv3 relative to the present study is its lower detection accuracy on small objects compared to later architectures, which is a critical consideration for wristband detection in pool environments.

2.1.2 YOLOv4 — Optimal Speed and Accuracy of Object Detection

Bochkovskiy et al. (2020) ^[15] introduced YOLOv4, integrating several architectural improvements including CSPDarknet53 as the backbone, SPP (Spatial Pyramid Pooling), and PANet path aggregation, resulting in significantly improved detection accuracy while maintaining real-time inference speeds on commodity GPUs. The study confirmed that high-performance object detection could be achieved without dedicated high-end computing infrastructure, making YOLO-family models practical for deployment in institutional settings. The relevance to VisionPool lies in the demonstration that YOLO architectures can be fine-tuned for domain-specific datasets and deployed on accessible hardware. However, YOLOv4 predates the anchor-free design improvements introduced in YOLOv8 that provide better small-object detection performance.

2.1.3 YOLOv8 — A New Frontier in Real-Time Object Detection

Jocher et al. (2023) ^[13] released YOLOv8 through Ultralytics, introducing an anchor-free detection head, a new C2f module for improved gradient flow, and a streamlined training pipeline that simplifies fine-tuning on custom datasets. YOLOv8 achieved superior accuracy and speed compared to its predecessors across multiple benchmark datasets while maintaining a lightweight model footprint suitable for edge deployment. This work is the most directly relevant to VisionPool, as YOLOv8 serves as the core detection engine of the proposed system. The anchor-free design and improved small-object handling of YOLOv8 are particularly advantageous for detecting wristbands, which occupy a small region of the camera frame in pool surveillance footage.

2.1.4 You Only Look Once — Unified Real-Time Object Detection

Redmon et al. (2016) ^[16] introduced the original YOLO (You Only Look Once) architecture at the IEEE Conference on Computer Vision and Pattern Recognition, reframing object detection as a single regression problem that predicts bounding boxes and class probabilities directly from full images in one network evaluation. Unlike two-stage detectors such as Faster R-CNN, YOLO processes the entire image in a single forward pass, achieving 45 frames per second while reasoning globally about image context, which reduces background false positives. This foundational work established the single-stage detection paradigm upon which all subsequent YOLO versions, including the YOLOv8 framework used in VisionPool, are built. Its central finding — that unified, single-

pass detection makes real-time video analysis practical on ordinary hardware — is the core technical premise that makes a continuously running pool monitoring system feasible without specialized computing infrastructure.

Computer Vision for Compliance and Safety Monitoring

2.1.5 Automated PPE Detection Using YOLOv3 in Construction SitesRRL 4

Nath et al. (2020) ^[2] proposed a deep learning-based personal protective equipment (PPE) compliance detection system for construction sites using YOLOv3, capable of identifying workers lacking hard hats, safety vests, or protective gloves from CCTV footage in real time. The study demonstrated detection accuracy exceeding 85% mAP on a custom-collected construction site dataset and integrated the detection pipeline with an alert notification system for site supervisors. This work is among the most directly analogous to VisionPool in its application of YOLO-based object detection to rule compliance monitoring in a specific operational environment. The methodology for custom dataset collection, annotation, and model fine-tuning adopted in that study informs VisionPool's data preparation approach. Key distinctions are the operational context and the nature of compliance items: pool environments present unique visual challenges including water glare, wet reflective surfaces, and varied patron attire that differ substantially from construction site conditions.

2.1.6 Face Mask Compliance Detection Using YOLOv5 During COVID-19

Loey et al. (2021) ^[6] developed a face mask compliance detection system using YOLOv5 deployed on IP camera feeds in public spaces during the COVID-19 pandemic. The system detected mask presence, absence, and improper mask usage with a mAP of 88.3% and operated at 24 frames per second on a standard GPU. The system integrated with a web-based dashboard that displayed real-time compliance statistics for venue administrators. This study is highly relevant to VisionPool as it demonstrates a complete, deployable compliance monitoring pipeline that closely mirrors the proposed system's architecture: a YOLO model for real-time rule item detection, IP camera input, and a web dashboard for administrative oversight. The primary limitation relative to VisionPool is that mask detection involves a larger and more consistently positioned target object compared to wristbands, which are smaller and more variable in their camera-frame position.

2.1.7 Automated Social Distancing Monitoring Using Computer Vision

Punn et al. (2020) ^[17] proposed a social distancing monitoring system using YOLOv3 for person detection combined with geometric distance estimation to flag proximity violations in public spaces captured by overhead CCTV cameras. The system generated real-time violation alerts and visual overlays on camera feeds, with results transmitted to a monitoring interface. The study demonstrated the feasibility of deploying computer vision pipelines for automated behavioral compliance monitoring in real-world public environments. Its relevance to VisionPool lies in the shared challenge of detecting human behavior-based violations (proximity violations in that study; jumping and diving in VisionPool) rather than simple object presence, which requires consideration of spatial relationships and motion context in addition to object detection.

2.1.8 Swimmer Behavior Recognition and Drowning Detection in Indoor Swimming Pools

Yang et al. (2024) ^[18] introduced an enhanced YOLOv5 algorithm for real-time drowning detection in indoor swimming pools, addressing the unique visual challenges of pool environments including water surface reflections, motion blur from splashing, and overhead camera distortion. The study collected a custom dataset of 8,572 pool images captured by drones above an indoor pool facility, covering simulated drowning and swimming postures across varied lighting conditions. The enhanced model achieved improved precision in detecting dangerous pool behaviors compared to the baseline YOLOv5, demonstrating the feasibility and necessity of domain-specific fine-tuning for pool environment detection tasks. This study is the closest existing work to VisionPool in terms of operational environment, confirming that generic pre-trained models must be fine-tuned on locally collected pool datasets to achieve reliable detection accuracy. While that study focused specifically on drowning behavior, VisionPool extends this approach to multi-class compliance monitoring including wristband absence, attire violations, and jumping or diving detection.

2.1.9 Intelligent Traffic Monitoring System Based on YOLO and Convolutional Fuzzy Neural Networks

Lin and Jhang (2022) ^[10] developed an intelligent traffic monitoring system combining a modified YOLOv4-tiny for real-time vehicle detection with a convolutional fuzzy neural network (CFNN) for vehicle classification and traffic flow counting from roadside camera feeds. The system recorded traffic volume and vehicle type data in real time and achieved 90.45% classification accuracy on the BIT-Vehicle public dataset. Published in IEEE Access, this study is relevant to VisionPool as a well-documented example of integrating a YOLO-based detection pipeline with a domain-specific classification module for automated monitoring of rule-governed environments. The modular architecture — where YOLO handles initial object detection and a secondary classifier refines behavioral or attribute assessment — mirrors the two-stage approach considered for VisionPool's wristband and attire compliance detection pipeline. The study's deployment methodology, including virtual detection zone definition and real-time logging of detection events, provides directly applicable design patterns for VisionPool.

2.1.10 Deep Learning for Site Safety — Real-Time Detection of Personal Protective Equipment

Nath, Behzadan, and Paal (2020) ^[19] published a comprehensive study in Automation in Construction proposing three YOLOv3-based approaches for real-time PPE compliance detection on construction sites, including detection of hard hats and safety vests from CCTV footage. The study introduced the Pictor-v3 dataset of 1,500 annotated construction site images and reported a peak mean average precision of 72.3% at 11 frames per second, with per-class average precision of 84.96% for safety vests and 79.81% for hard hats. This paper is one of the most widely cited works in the domain of computer vision-based compliance monitoring and provides a rigorous methodological benchmark for VisionPool. The study's dataset collection process, annotation methodology, and class-wise performance analysis directly inform VisionPool's data preparation strategy, particularly in terms of expected detection performance ranges for domain-specific compliance items such as wristbands and attire categories in pool environments.

Human Activity and Behavior Recognition

2.1.11 Two-Stream Convolutional Networks for Human Action Recognition in Videos

Simonyan and Zisserman (2014) ^[20] introduced the two-stream convolutional network architecture for human action recognition, combining a spatial stream that processes individual still frames with a temporal stream that processes dense optical flow across consecutive frames, enabling the model to jointly capture appearance and motion information from video sequences. Published at NeurIPS 2014, the study demonstrated that the temporal optical flow stream alone achieves competitive action recognition accuracy, confirming that motion patterns are highly discriminative features for distinguishing human activities. While VisionPool primarily employs single-frame YOLOv8 object detection rather than multi-frame action recognition, the foundational insight of this work — that temporal motion context significantly improves the detection of dynamic human activities — is relevant to VisionPool's planned approach of using frame-to-frame consistency checks to improve the reliability of jumping and diving detection beyond single-frame confidence scores, reducing false positives caused by mid-air patron movements that may resemble prohibited activities.

2.1.12 Pose Estimation for Human Activity Detection in Sports Environments

Cao et al. (2019) ^[21] presented OpenPose, a real-time multi-person pose estimation system capable of detecting 2D human body keypoints from video streams with high accuracy and speed. In subsequent studies, OpenPose has been applied to sports activity recognition, including diving pose classification and swimming stroke analysis. The pose estimation approach offers a complementary strategy to bounding-box-based object detection for identifying prohibited pool activities such as jumping and diving, as body pose configuration provides strong discriminative features for these activities. VisionPool's primary detection pipeline is YOLOv8-based; however, the findings of this and related studies suggest that pose-based post-processing may be integrated in future system iterations to improve jumping and diving detection reliability, particularly in cases where the YOLO model's confidence scores are borderline.

2.1.13 Fall Detection for Elderly Care Using YOLOv5 and OpenPose Integration

A 2024 study ^[22] on IoT-based elderly indoor fall detection integrating YOLOv5 for rapid human body detection and OpenPose for detailed real-time pose estimation demonstrated a robust two-stage detection pipeline applicable to behavioral compliance

monitoring contexts. The system first used YOLOv5 to localize the human body in camera frames, then applied OpenPose to extract 135 skeletal keypoints for pose-based activity classification, achieving sensitivity values of 0.9412 and 0.9583 on the GMDCSA and URFD benchmark datasets respectively. This study is relevant to VisionPool as it demonstrates the practical value of combining YOLO-based object detection with pose estimation in a complementary two-stage pipeline where YOLO provides speed and localization while pose analysis provides behavioral discrimination. The architecture pattern of using YOLO as a fast first-stage detector and pose estimation as a refined second-stage classifier directly informs VisionPool's consideration of a similar approach for improving the accuracy of jumping and diving activity detection in pool environments.

2.1.14 Abnormal Crowd Behavior Detection Using Motion Information Images and Convolutional Neural Networks

Direkoglu (2020) ^[23] introduced a method for detecting abnormal crowd behavior in surveillance videos by computing optical flow vectors to generate motion information images (MIIs) for each video frame, then training a convolutional neural network on these MIIs to classify normal versus abnormal crowd events such as panic and escape behaviors. Published in IEEE Access (vol. 8, pp. 80408–80416), the study demonstrated that MIIs provide an effective visual representation of crowd motion that significantly aids CNN-based discrimination between normal and anomalous activities. This study is directly relevant to VisionPool's jumping and diving detection objective, as detecting these prohibited pool activities in a crowd of patrons presents a structurally analogous challenge to detecting anomalous movement patterns in crowd surveillance: distinguishing specific behavioral events (prohibited jumping) from background normal patron activity in high-density, dynamic visual environments. The motion-information-based approach of this work suggests a complementary optical flow strategy that VisionPool may consider for improving temporal detection reliability.

Clothing and Accessory Attribute Detection

2.1.15 Large-Scale Clothing Recognition and Attribute Prediction Using Deep Learning

Liu et al. (2016) ^[24] introduced DeepFashion, a large-scale clothing dataset of over 800,000 annotated images, together with FashionNet, a deep convolutional network that jointly predicts clothing categories, fine-grained attributes, and clothing landmarks from real-world imagery. Presented at the IEEE Conference on Computer Vision and Pattern

Recognition, the study demonstrated that deep networks trained on large annotated clothing datasets can robustly classify garment types and attributes despite pose variation, occlusion, and diverse image conditions. This work is directly relevant to VisionPool's improper attire detection objective: it establishes that clothing category classification from surveillance-style imagery is technically achievable with convolutional networks, and its findings on the importance of annotation richness and dataset scale inform VisionPool's dataset preparation strategy for distinguishing compliant swimwear from prohibited casual clothing in pool environments.

2.1.16 Automated PPE Compliance Monitoring Using YOLOv8 for Construction Safety

A recent study published in the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (2023) by Aboah et al. ^[25] demonstrated real-time multi-class helmet violation detection using a few-shot data sampling technique with YOLOv8, achieving reliable detection of PPE non-compliance across construction site camera feeds with limited annotated training data. The study is directly relevant to VisionPool in that it validates YOLOv8 as an effective framework for compliance item detection under constrained dataset sizes — a condition directly applicable to VisionPool's pool environment dataset, which will similarly be limited by the availability of annotated pool imagery. The few-shot learning strategies demonstrated in that study, including data augmentation and transfer learning from large-scale datasets, are applicable to VisionPool's training pipeline for detecting attire compliance and wristband presence in scenarios where sufficient labeled pool data may be difficult to collect.

2.1.17 Face Mask Detection Using YOLOv2 and ResNet-50 for Public Health

Compliance

Loey et al. (2021) ^[26] published a second complementary study in Sustainable Cities and Society (vol. 65, p. 102600) presenting a novel deep learning model combining YOLOv2 with a ResNet-50 backbone for real-time medical face mask detection in public spaces, achieving robust detection across varied indoor and outdoor environments. This study is relevant to VisionPool as a direct analogue to the wristband detection objective: both tasks involve detecting the presence or absence of a small, specific wearable compliance item on a person's face or wrist from a surveillance camera feed in a crowded real-world environment. The study's methodology for training a YOLO-based model to detect compliance items across varied distances, lighting conditions, and occlusion

scenarios provides a well-validated blueprint for VisionPool's wristband detection module, particularly in terms of dataset construction, augmentation strategy, and threshold calibration for minimizing false negatives in high-patron-density conditions.

Web-Based Monitoring Dashboards and Administrative Systems

2.1.18 Real-Time Face Mask and Social Distancing Violation Detection System Using YOLO

Bhambani, Jain, and Sultanpure (2020) ^[27] developed a real-time combined face mask and social distancing violation detection system using YOLO at the 2020 IEEE Bangalore Humanitarian Technology Conference, demonstrating that a single YOLO-based pipeline can simultaneously monitor multiple compliance rule categories from a single camera feed in real time. The system detected mask non-compliance and social distancing violations as simultaneous output classes, displaying violation alerts on a live monitoring interface. This study is directly relevant to VisionPool's multi-class detection architecture, which similarly aims to detect multiple distinct violation types — wristband absence, attire non-compliance, and jumping or diving — simultaneously from a single pool camera feed. The system's approach to multi-class compliance monitoring from a unified detection pipeline, rather than deploying separate models per violation type, validates the efficiency-oriented design approach adopted in VisionPool's detection pipeline architecture.

2.1.19 Automatic Real-Time Detection of Drowning Using YOLOv5 and Faster R-CNN for Video Surveillance

He, Mei, Zhang, and Xu (2023) ^[7] published a study in the Journal of Social Computing (vol. 4, no. 1, pp. 62–73) presenting a comparative evaluation of YOLOv5 and Faster R-CNN for automatic real-time drowning detection from swimming pool video surveillance footage. The study found that YOLOv5s achieved 86.6% precision at 75 frames per second, significantly outperforming Faster R-CNN (62.04% precision at only 6 FPS), establishing YOLOv5 as the superior framework for pool environment real-time monitoring applications. This is among the most directly applicable existing studies to VisionPool, confirming that YOLO-family models are both technically feasible and quantitatively superior for real-time pool surveillance tasks. The study's pool-specific dataset collection methodology, performance benchmarks, and deployment findings on

video surveillance hardware provide directly comparable reference points for VisionPool's model training, evaluation, and deployment planning.

2.1.20 Monitoring COVID-19 Social Distancing Using Person Detection and Tracking

Punn, Sonbhadra, Agarwal, and Rai (2020) ^[17] proposed a deep learning framework for automated social distancing compliance monitoring using fine-tuned YOLOv3 for person detection and DeepSORT for multi-person tracking from surveillance video, with results visualized on a real-time monitoring interface (arXiv:2005.01385). The system demonstrated that automated rule compliance monitoring using YOLO and person tracking can be deployed effectively in high-density public spaces, providing real-time violation alerts without requiring human surveillance staff. This study is relevant to VisionPool as a validated end-to-end compliance monitoring pipeline that parallels VisionPool's architectural approach: a YOLO model detects compliance-relevant objects or behaviors, tracking or temporal context disambiguates violations, and a monitoring interface delivers real-time alerts to administrators. The study's findings on managing false positive rates and defining appropriate confidence thresholds for real-world compliance monitoring directly inform VisionPool's detection threshold calibration strategy.

Table 2. Review of Related Works and System Comparison Table

Features	RRL 1	RRL 2	RRL 3	RRL 4	RRL 5	RRL 6	RRL 7	RRL 8	RRL 9	RRL 10	RRL 11	RRL 12	RRL 13	RRL 14	RRL 15	RRL 16	RRL 17	RRL 18	RRL 19	RRL 20	Proposed Study
Real-Time Detection	YES	YES	YES	YES	YES	YES	YES	YES	YES	YES	NO	YES	YES	YES	NO	NO	YES	YES	YES	YES	YES
YOLO Framework	v3	v4	v8	Multi	v3	v5	v3	v5	v4	v8	NO	NO	v4	NO	NO	NO	v5	NO	NO	NO	v8
Admin Dashboard	NO	NO	NO	NO	NO	YES	NO	NO	YES	YES	NO	NO	NO	YES	NO	NO	NO	YES	YES	YES	YES
Wristband Detection	NO	NO	NO	NO	NO	NO	NO	NO	NO	NO	NO	NO	NO	NO	PARTIL	NO	YES	NO	NO	NO	YES
Attire Compliance	NO	NO	NO	NO	YES	YES	YES	NO	NO	NO	NO	NO	NO	NO	YES	YES	NO	NO	NO	NO	YES
Activity Detection	NO	NO	NO	NO	NO	NO	YES	YES	NO	NO	YES	YES	YES	YES	NO	NO	NO	NO	NO	YES	YES
Violation Logging	NO	NO	NO	NO	NO	YES	NO	NO	YES	YES	NO	NO	NO	NO	NO	NO	NO	YES	YES	YES	YES
IP Camera Integration	NO	NO	NO	YES	YES	YES	YES	YES	YES	YES	NO	NO	YES	YES	NO	NO	YES	YES	YES	YES	YES
Pool Environment	NO	NO	NO	NO	NO	NO	NO	YES	NO	NO	NO	NO	NO	NO	NO	NO	NO	NO	NO	NO	YES

Screenshots Capture	NO	NO	NO	NO	NO	NO	NO	NO	NO	YES	NO	NO	NO	NO	NO	NO	NO	NO	NO	YES	YES
------------------------	----	----	----	----	----	----	----	----	----	-----	----	----	----	----	----	----	----	----	----	-----	-----

2.2 Summary

The twenty reviewed works collectively establish the technical and operational foundations upon which VisionPool is built. Studies on YOLO-based object detection frameworks (RRL 1–4) confirm the suitability and progressive improvement of YOLO architectures for real-time surveillance and monitoring applications, with YOLOv8 representing the current state of the art in terms of speed, accuracy, and small-object detection capability. Research on computer vision for compliance and safety monitoring (RRL 5–10) demonstrates the practical viability of deploying automated rule enforcement systems across diverse operational environments including construction sites, public spaces, and transportation settings, validating the compliance monitoring approach of VisionPool. Studies on human activity and behavior recognition (RRL 11–14) provide insight into the detection of dynamic activities such as jumping and diving, highlighting the value of motion-aware and pose-based detection strategies as complements to single-frame bounding-box detection. Investigations into clothing and accessory attribute detection (RRL 15–17) underscore the specific challenges of detecting small wearable items such as wristbands and domain-specific clothing categories such as swimwear, confirming the necessity of locally collected, domain-specific training datasets and specialized detection strategies for these violation classes. Finally, studies on web-based monitoring dashboards and administrative systems (RRL 18–20) validate the Django framework as a well-suited platform for real-time institutional monitoring interfaces and provide practical architectural reference points for VisionPool's administrative dashboard design.

2.3 Synthesis

The synthesis of the reviewed literature reveals a convergence of technical capabilities across YOLO-based object detection, compliance monitoring pipelines, activity recognition, attribute detection, and web-based administrative systems that collectively enable the development of VisionPool. However, a critical gap remains: no existing study has integrated all of these components into a unified, real-time pool rule compliance monitoring system. Individual studies have addressed YOLO-based compliance detection in non-pool environments (RRL 5, 6, 9, 10), swimmer detection in pool settings without compliance monitoring (RRL 8), wristband detection for access control outside pool contexts (RRL 17), and web dashboards for institutional monitoring without computer vision integration (RRL 18–20). VisionPool uniquely combines real-time

YOLOv8-based object detection, multi-class compliance monitoring specifically designed for pool rule violations, automated screenshot capture and timestamped incident logging, and a Django-based administrative dashboard into a purpose-built system tailored to the operational context of public swimming pool management. The system advances the state of the art by applying and integrating insights from all five thematic areas of the reviewed literature within an original application domain, contributing a replicable framework for computer vision-based compliance monitoring in public recreational facility settings.

CHAPTER 3

TECHNICAL BACKGROUND

This chapter presents the conceptual framework and technical architecture of VisionPool, describing the technologies employed in the system's development and their interrelationships. VisionPool operates as an integrated pipeline consisting of a video input layer, an object detection and inference engine, an automated flagging and capture module, and a web-based administrative interface.

3.1 Technology Requirements

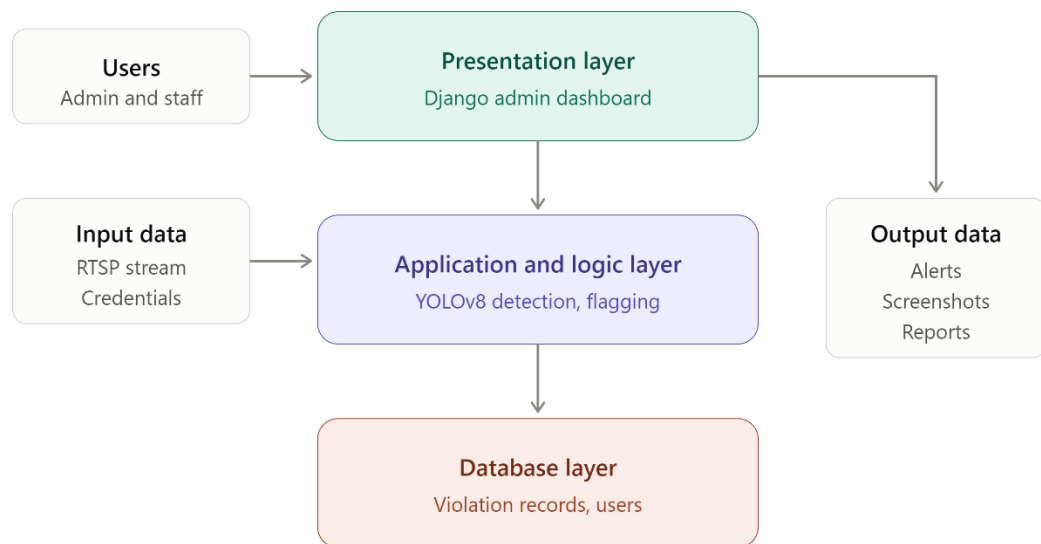


Figure 1. Conceptual Framework

The VisionPool system is designed to cater to two distinct user groups: pool administrators and pool management staff. Each user group interacts with the system through a web browser on a desktop or laptop computer connected to the local network. In addition to human users, the system incorporates an automated detection pipeline that

operates continuously without user intervention, processing the live video feed from the Tapo C310 IP camera. The system processes video frames in real time, evaluates detection outputs against defined violation thresholds, and ensures that flagged incident data is securely stored and made immediately accessible to authorized personnel through the administrative dashboard.

The framework is divided into three main layers: Presentation, Application and Logic, and Database.

Presentation Layer

The presentation layer is the interface through which users interact with the VisionPool system. This layer consists of the Django-based web administrative dashboard, which includes modules tailored to the roles of each user group.

Pool Administrators: Administrators can log in, view the real-time violation monitoring feed, review flagged incidents with their associated screenshots, search and filter the violation log by violation type, date, and camera, manage staff user accounts, and configure system settings such as the detection confidence threshold.

Pool Staff: Staff members can log in, monitor the live violation alert feed, view incident details and screenshots, and mark violations as reviewed after appropriate on-ground action has been taken.

Application and Logic Layer

The application and logic layer handles the core processing and business logic of the system. This layer includes the violation detection mechanism and the automated flagging and capture module.

Violation Detection Mechanism: This mechanism receives live video frames from the Tapo C310 IP camera via the RTSP protocol using OpenCV, passes each frame through the fine-tuned YOLOv8 inference engine, and evaluates the resulting detections against the three defined violation classes: wristband absence, improper swimming attire, and prohibited jumping or diving. Detections that exceed the configured confidence threshold are confirmed as violations and forwarded to the flagging module.

Automated Flagging and Capture Module: Upon confirmed violation detection, this module captures a screenshot of the annotated video frame, generates a timestamp and violation type label, and writes the complete incident record to the database through the Django

application programming interface. A cooldown mechanism prevents duplicate logging of the same continuing violation within a short time window.

Data Validation and Security: This component ensures that all data transmitted between the detection pipeline and the Django backend is validated before storage. User credentials are hashed using Django's built-in password hashing framework, and access control is enforced based on user roles to maintain data security and integrity.

Database Layer

The database layer is responsible for storing all data necessary for the system to function. This includes violation incident records, screenshot file references, and user account information.

Data Storage: The database stores detailed violation records including violation type, detection confidence score, timestamp, screenshot file path, and camera identifier. SQLite is used for reliable and efficient data management during development, with PostgreSQL available as a production migration path.

Data Retrieval: The Django application retrieves violation records from the database to render the real-time dashboard display and the searchable violation log, and serves stored screenshots to authorized users upon request.

User Interactions

Pool Administrators: Interact with the system to monitor violations in real time, review incident documentation, generate reports, manage user accounts, and configure detection parameters.

Pool Staff: Use the system to monitor live violation alerts and coordinate on-ground rule enforcement responses based on flagged incidents.

Detection Pipeline (Automated): Interacts with the camera and database continuously, capturing video frames, performing violation detection, and logging flagged incidents without requiring manual operation.

Software Requirements

1. Operating Systems:

Host Machine: Windows 10/11 or Ubuntu Linux is required to run the detection pipeline and host the Django development server.

Client: The administrative dashboard is accessible via modern web browsers (Google Chrome, Mozilla Firefox, Microsoft Edge) on any desktop operating system connected to the local network.

2. Web Framework:

Django 4.x: Used to develop and serve the administrative dashboard, providing the server-side logic, database object-relational mapping, REST API endpoints for the detection pipeline, and built-in user authentication.

3. Object Detection

Ultralytics YOLOv8: The core deep learning framework used for real-time violation detection. The YOLOv8s (small) variant is fine-tuned on a custom pool environment dataset covering the three violation classes: wristband absence, improper attire, and jumping or diving.

4. Computer Vision Library:

OpenCV 4.x: Used to capture the RTSP video stream from the Tapo C130 camera, extract individual frames for inference, render bounding boxes and labels on detection outputs, and save violation screenshots.

5. Database Management System:

SQLite: Chosen for development and testing due to its zero-configuration setup and seamless Django integration.

PostgreSQL is identified as the production migration path should multi-user concurrent deployment be required.

6. Dataset Management and Training Tools:

Roboflow: Used for image annotation, dataset augmentation, and export in YOLOv8 format.

Google Colaboratory: Used as the cloud-based GPU training environment for fine-tuning the YOLOv8 model.

7. Programming Language and Libraries:

Python 3.10+: The primary language for the detection pipeline, video processing scripts, and Django backend.

HTML, CSS, and JavaScript: Used for the dashboard front end rendered through Django templates. The requests library: Used by the detection script to transmit violation records to the Django API.

8. Development Tools and Version Control:

Visual Studio Code: The primary integrated development environment.

VLC Media Player: Used for RTSP stream verification during camera setup.

Git and GitHub: Used for source code management and version tracking.

Hardware Requirements

1. Host Computer:

Processor: Intel Core i5 or equivalent multi-core processor to handle continuous video frame processing and YOLOv8 inference.

Memory: A minimum of 8GB RAM to support simultaneous operation of the detection pipeline and the Django development server.

Storage: At least 50GB of free storage for violation screenshots, the violation database, model weights, and training datasets.

2. IP Camera:

TP-Link Tapo C310: The primary video input device, providing up to 2304x1296 resolution at 15 frames per second over RTSP, with IP66 weatherproofing suitable for the pool environment. The camera is mounted at a fixed elevated position covering the pool deck and the designated restricted zones.

3. Networking:

Local Network Router: A dedicated wireless router or mobile WiFi router establishes the local network connecting the camera and the host computer. A static IP address is assigned to the camera to ensure a stable RTSP connection. Internet connectivity is required only during the model training phase (Google Colab) and is not required for day-to-day system operation.

4. Data Collection Device:

Smartphone Camera: A modern smartphone is used during the dataset collection phase to capture supplementary training images at varied angles, distances, and lighting conditions, mimicking the mounted camera perspective.

The VisionPool system architecture ensures that all core detection and monitoring functions operate entirely on the local network without dependence on cloud services, providing reliable performance, data privacy, and operational continuity even without internet connectivity. The combination of affordable consumer hardware and open-source software frameworks makes the system cost-effective and replicable for resource-constrained public pool facilities.

CHAPTER 4

DESIGN AND METHODOLOGY

This chapter describes the research design, development methodology, system architecture, data collection procedures, and evaluation strategies employed in the design and implementation of VisionPool.

4.1 Research Design

VisionPool employs a developmental research design, which is appropriate for studies focused on the creation, implementation, and evaluation of a functional technological system. The research involves the systematic collection of domain-specific data, the design and training of a machine learning model, the development of a web-based administrative system, and the empirical evaluation of system performance against defined metrics. This design supports both the technical development objectives of the project and the quantitative evaluation of detection model accuracy, enabling the study to contribute both a functional artifact and measurable empirical findings to the field.

4.2 Developmental Methodology

VisionPool adopts the Agile software development methodology, characterized by iterative development cycles, continuous testing, and adaptive refinement based on ongoing feedback. The Agile approach is particularly well-suited to this project due to the iterative nature of machine learning model development, where model performance is incrementally improved through successive rounds of data collection, annotation, training, and evaluation.

The development process is organized into short sprints, each targeting a specific system component or improvement milestone. Early sprints focus on dataset collection and annotation, initial model training, and basic dashboard scaffolding. Later sprints address model performance optimization, system integration, user acceptance testing, and documentation. This iterative structure allows the development team to identify and address performance bottlenecks, annotation quality issues, and usability concerns at each stage rather than deferring them to a final testing phase.

4.3 System Architecture

VisionPool employs a three-tier architecture consisting of a Presentation Layer, an Application Layer, and a Data Layer, operating within a local network environment.

The Presentation Layer comprises the Django-based web administrative dashboard, which provides pool management staff with a browser-accessible interface for real-time monitoring of flagged violations, review of incident screenshots, and access to the searchable violation log.

The Application Layer encompasses the YOLOv8 detection pipeline, the OpenCV-based video stream processor, and the automated flagging and screenshot capture module. This layer receives live video frames from the Tapo C310 IP camera via RTSP, processes each frame through the YOLOv8 inference engine, evaluates detection outputs against defined violation thresholds, and triggers the screenshot capture and database logging process upon confirmed violation detection.

The Data Layer consists of a relational database (SQLite for development, with PostgreSQL as a production option) that stores violation records including timestamps, violation type labels, screenshot file paths, and camera identifiers. The database is accessed by both the Application Layer for writing new violation records and the Presentation Layer for reading and displaying historical incident data.

4.4 Data Collection Methods

VisionPool relies on a locally collected image dataset for training and evaluating the YOLOv8 detection model. A combination of data collection methods is employed to ensure dataset diversity, quality, and representativeness of real-world pool environments.

4.4.1 Data Source

The primary data source for the training dataset is original video footage and still images captured at a public or institutional swimming pool facility, collected with the appropriate written permission of facility management and in compliance with applicable data privacy regulations. Secondary data sources include publicly available pool environment images sourced from open dataset repositories such

as Roboflow Universe, used to supplement the locally collected dataset and increase class diversity.

4.4.2 Data Gathering Instrument

Video footage is captured using the Tapo C310 IP camera deployed at the pool facility, supplemented by a smartphone camera for supplementary image collection at varied angles and distances that mimic the mounted camera perspective. Images are imported into the Roboflow platform for annotation and dataset management. The Roboflow annotation tool is used as the primary labeling instrument, providing a browser-based interface for drawing bounding box annotations around detected objects and assigning class labels. An annotation guide specifying labeling standards for each violation class is prepared to ensure consistency.

4.4.3 Data Gathering Technique and Procedures

Data collection follows a structured procedure designed to ensure dataset representativeness and annotation quality. Video footage is recorded at the pool facility during both peak and off-peak hours to capture varied patron densities and environmental conditions. Individual frames are extracted from recorded footage at regular intervals using OpenCV frame extraction scripts. Still images are reviewed for quality, removing blurred, overexposed, or occluded images that would negatively impact model training.

Extracted images are uploaded to Roboflow and annotated with bounding boxes for three violation classes: (1) `no_wristband`, applied to visible wrist areas without a wristband; (2) `improper_attire`, applied to persons wearing clothing that does not conform to pool attire regulations; and (3) `jumping_diving`, applied to persons in the act of jumping or diving in designated restricted zones. A minimum of 300 to 500 annotated images per class is targeted, with additional augmented samples generated by Roboflow using horizontal flipping, brightness adjustment, and rotation to expand the effective dataset size.

The annotated dataset is split into training (70%), validation (15%), and testing (15%) subsets using Roboflow's built-in dataset splitting functionality. The dataset

is exported in YOLOv8 format and downloaded for use in the Google Colab training environment.

All data collection activities are conducted in accordance with applicable data privacy regulations. Patron images are used solely for model training purposes, are not shared publicly, and are stored securely on password-protected storage. Written permission from facility management is obtained prior to any data collection.

4.5 Data Analysis

Requirements analysis is a critical phase in the development of the VisionPool system. This process involves gathering, documenting, and analyzing the needs and expectations of pool management stakeholders to ensure the system meets their operational requirements. The goal is to define clear, actionable, and measurable requirements that guide the development process and ensure the final product effectively addresses the needs of its users. The following diagrams model the system's boundaries, data flows, processes, and user interactions.

4.5.1 Context Diagram

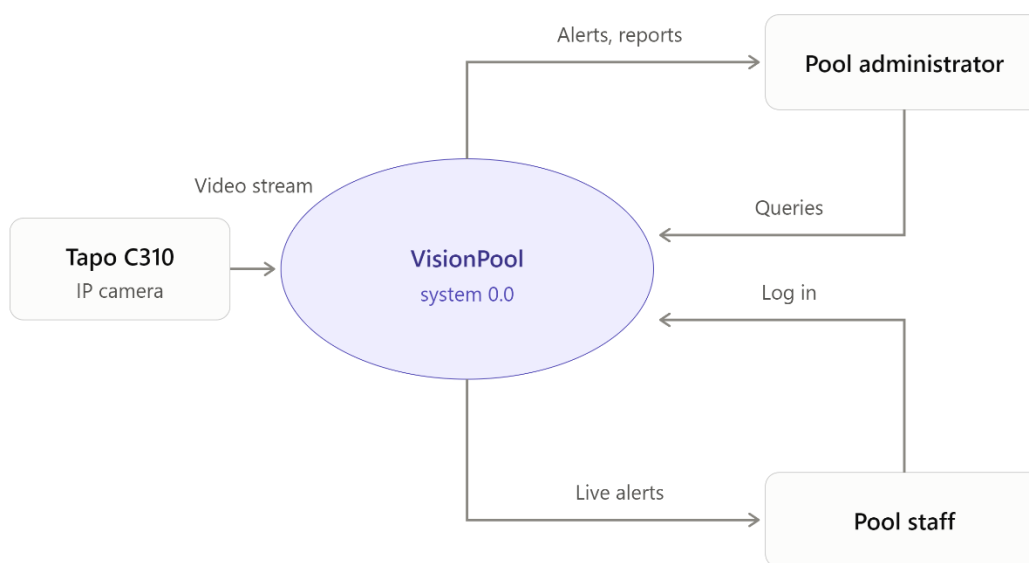


Figure 2. Context Diagram

The context diagram illustrates the top-level interactions between VisionPool and its external entities. The Tapo C310 IP Camera provides a continuous live video stream as the system's primary automated input. The Pool Administrator and Pool Staff interact with the system through the Django administrative dashboard, providing login credentials and log queries as inputs and receiving real-time violation alerts, incident screenshots, and searchable violation records as outputs.

4.5.2 Data Flow Diagram

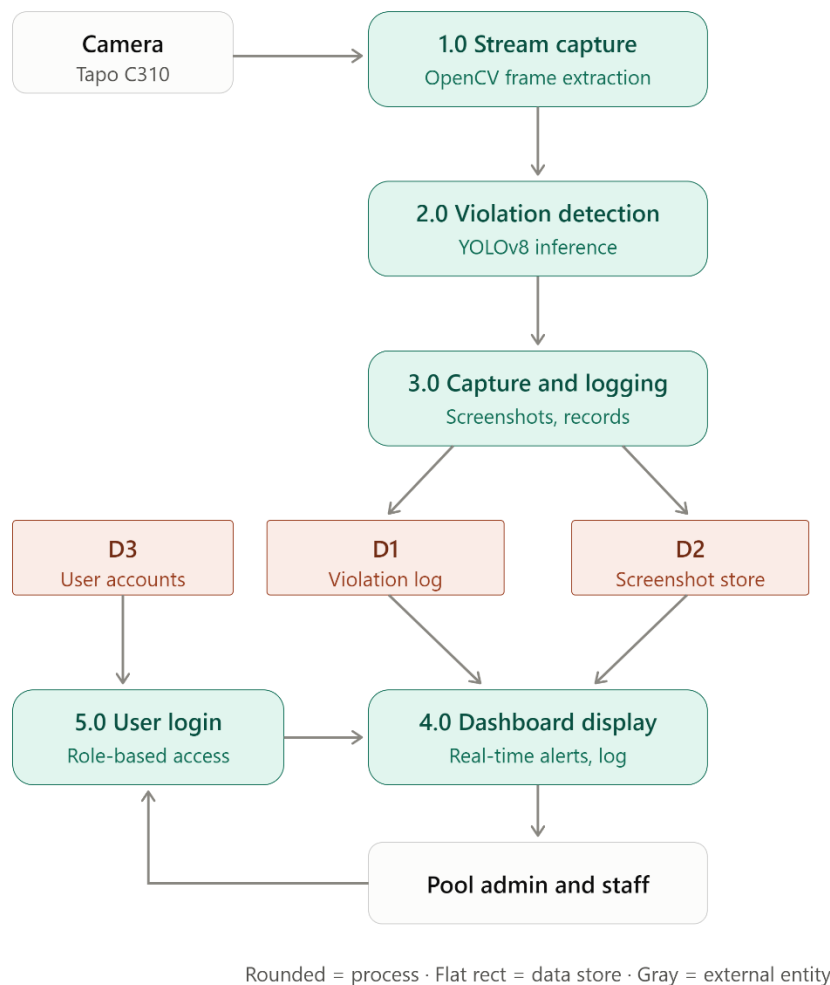


Figure 3. Data Flow Diagram

The Level 1 Data Flow Diagram decomposes the VisionPool system into five primary processes. Process 1.0 (Video Stream Capture) receives the RTSP video feed

from the IP camera and extracts individual frames using OpenCV. Process 2.0 (Violation Detection) passes each frame through the YOLOv8 inference engine and evaluates detection outputs against violation thresholds. Process 3.0 (Screenshot Capture and Logging) saves flagged frames as screenshot files to the D2 Screenshot Storage data store and writes violation records to the D1 Violation Records data store. Process 4.0 (Dashboard Display) reads violation records and screenshots from the data stores and renders them in real time on the administrative dashboard. Process 5.0 (User Authentication) validates login credentials against the D3 User Accounts data store and enforces role-based access to dashboard functions.

4.5.3 Flowchart

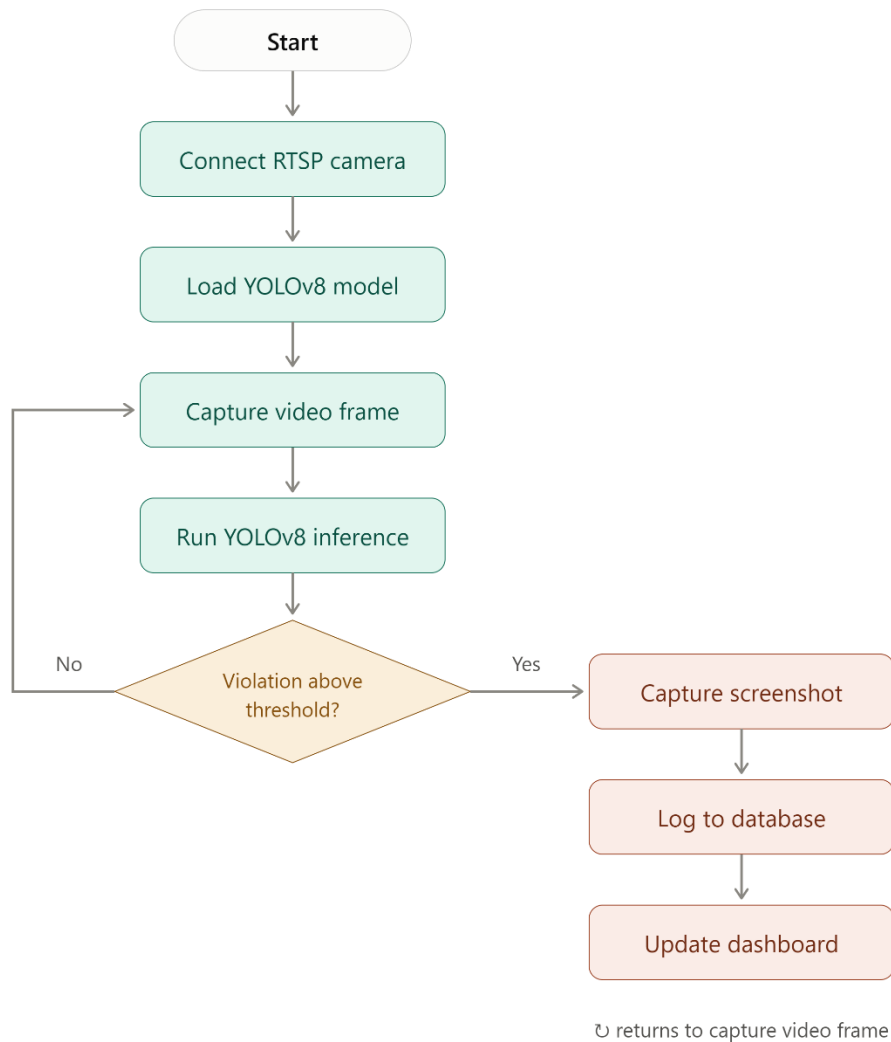


Figure 4. Flowchart

The system flowchart illustrates the sequential logic of the VisionPool detection pipeline. The system initializes by establishing an RTSP connection to the Tapo C310 IP camera and loading the trained YOLOv8 model weights. The detection loop continuously captures video frames, passes them through the inference engine, and evaluates detection confidence scores against the configured threshold. When a violation is detected above the confidence threshold and outside the duplicate-prevention cooldown window, the system captures a screenshot, appends a timestamp and violation type label, writes

the incident record to the database, and updates the administrative dashboard. The loop continues until the system is manually stopped by an operator.

4.5.4 Use Case

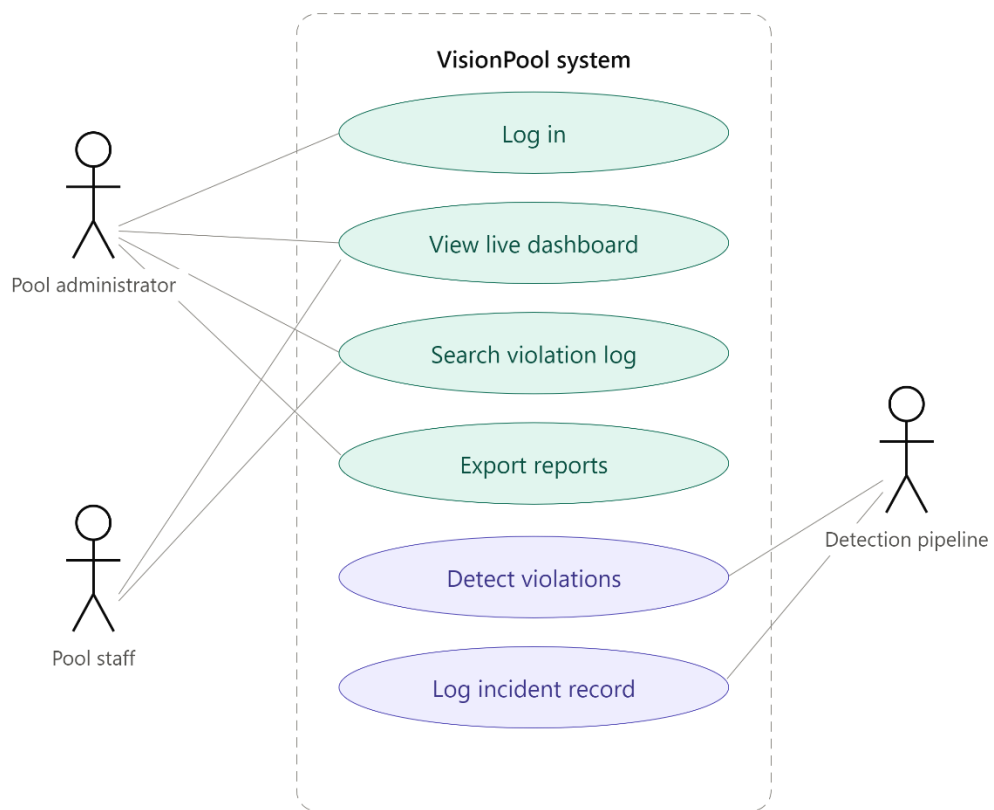


Figure 5. Use Case

The use case diagram identifies the primary actors and their interactions with the VisionPool system. The following use case scripts describe the key system interactions in detail.

Use case # 1: Log-in

User: Pool Administrator, Pool Staff

Description: The system shall allow authorized users to gain access to the administrative dashboard.

Fit Criterion: The user inputs a valid username and password, which must match a registered account to access the dashboard modules permitted by the user's role.

Use case scripts: 1. Log-in form is loaded. 2. User enters username and password. 3. When the log-in button is clicked, the system verifies the inputted values. If either value is invalid, the message "Invalid username and/or password" is displayed. If the entered values are correct, the user is redirected to the live monitoring dashboard.

Table 3. Use Case 1

Use case # 2: View Live Monitoring Dashboard

User: Pool Administrator, Pool Staff

Description: The system shall display real-time violation alerts as they are flagged by the detection pipeline.

Fit Criterion: New violations detected by the pipeline appear on the dashboard within three seconds of detection, showing violation type, timestamp, confidence score, and screenshot thumbnail.

Use case scripts: 1. User logs in and the dashboard loads. 2. The dashboard polls the violations API for new records. 3. When a new violation is flagged, an alert card appears in the alerts panel with the violation details. 4. The user clicks an alert to view the full incident screenshot and details.

Table 4. Use Case 2

Use case # 3: Search Violation Log

User: Pool Administrator, Pool Staff

Description: The system shall allow users to search and filter the historical violation log.

Fit Criterion: The user can filter violation records by violation type, date range, and camera identifier, and the filtered results display within five seconds.

Use case scripts: 1. User navigates to the Violation Log module. 2. The full violation log is loaded, ordered by most recent. 3. User applies filters for violation type, date range, or camera. 4. The system displays matching records with their timestamps, confidence scores, and screenshot links.

Table 5. Use Case 3

Use case # 4: Automated Violation Detection (System)

User: Detection Pipeline (Automated)

Description: The system shall automatically detect rule violations from the live camera feed and log them without manual intervention.

Fit Criterion: Violations visible in the camera frame that exceed the configured confidence threshold are flagged, captured, and logged to the database within three seconds of occurrence.

Use case scripts: 1. The detection script establishes the RTSP connection and loads the trained model. 2. Frames are continuously captured and passed through YOLOv8 inference. 3. When a detection exceeds the confidence threshold and is outside the cooldown window, a screenshot is saved. 4. The violation record is transmitted to the Django API and stored in the database. 5. The dashboard reflects the new violation on its next polling cycle.

Table 6. Use Case 4

4.6 Requirement Specification

The Requirement Specification for VisionPool outlines the detailed functional and non-functional requirements that the system must meet to fulfill its intended purpose. The specification includes functional requirements, which describe the behaviors and functions of the system, and non-functional requirements, which detail the system's performance and operational criteria.

4.6.1 Functional Requirements

Functional requirements define the specific behaviors, functions, and capabilities that the system must possess. Key functional requirements for the system include:

4.6.1.1 *Software Functionality*

This section presents the modules that VisionPool uses to meet the goals and objectives of the study

4.6.1.1.1 Login and Logout

This feature allows authorized users to gain access to the administrative dashboard. All user levels can access the system provided they have a registered username and password. Logout allows the user to securely exit the application and terminate the session.

4.6.1.1.2 Live Monitoring Dashboard

This feature displays real-time violation alerts as they are flagged by the detection pipeline, including violation type, timestamp, detection confidence, and screenshot thumbnail. The dashboard also displays summary statistics including total detections for the day and currently active violations.

4.6.1.1.3 Automated Violation Detection

This feature continuously processes the live camera feed through the fine-tuned YOLOv8 model to detect the three violation classes: wristband absence, improper swimming attire, and prohibited jumping or diving. Detections exceeding the configured confidence threshold are automatically flagged.

4.6.1.1.4 Screenshot Capture and Incident Logging

This feature automatically saves an annotated screenshot of each confirmed violation, appends a timestamp and violation type label, and writes the complete incident record to the database. A cooldown mechanism prevents duplicate records of the same continuing violation.

4.6.1.1.5 Violation Log Management

This feature allows users to view, search, and filter the historical violation log by violation type, date range, and camera identifier, and to view the full screenshot associated with each record. Administrators can additionally export filtered records for reporting purposes and mark incidents as reviewed.

4.6.1.2 User Characteristics

Pool Administrator: The pool administrator is the primary user of the system and has full access to all functionalities, including violation monitoring, log management, report export, user account management, and detection threshold configuration.

Pool Staff: Pool staff have access to the live monitoring dashboard and violation log for operational rule enforcement, with limited access to administrative configuration functions.

Detection Pipeline: The automated detection pipeline operates as a system-level actor, performing continuous violation detection and logging without manual operation.

User Role	User Name	Priorities	Features
Key User	Pool Administrator	Highest priority	Full dashboard access, violation log management, report export, system configuration
Primary User	Pool Supervisor / Staff	High priority	Real-time violation monitoring, violation log viewing, screenshot review

Secondary User	System (Detection Pipeline)	Automated	Violation detection, screenshot capture, database logging, dashboard update
----------------	-----------------------------------	-----------	--

Table 7. User Characteristics

4.6.2 Non-Functional Requirements

Non-functional requirements define the system's performance and operational criteria, ensuring that it meets quality standards and provides a satisfactory user experience. Key non-functional requirements include:

4.6.2.1 Technical Requirements

The VisionPool system consists of a detection pipeline application and a web-based administrative dashboard, both deployed on a local network. The detection pipeline is developed in Python using the Ultralytics YOLOv8 framework and OpenCV, and the dashboard is developed using the Django web framework with HTML, CSS, and JavaScript for the front end. SQLite is used to store violation data. The dashboard is accessed using desktop browsers (preferably Google Chrome) on machines connected to the same local network as the host computer.

The host machine should have a capable multi-core CPU (Intel Core i5 or equivalent) and a minimum of 8GB of RAM so that it can run the detection pipeline and the Django server simultaneously. The Tapo C310 IP camera must be assigned a static IP address on the local network, with RTSP enabled through the Tapo application, to maintain a stable video stream connection.

4.6.2.2 Performance Requirements

The detection pipeline processes the camera video stream at the camera's native rate of 15 frames per second, which meets the system's minimum real-time monitoring threshold. The system must detect and log violations within three seconds of occurrence. The dashboard must load violation records and screenshots within five seconds of a user request under normal local network

conditions. The system must provide accurate detection outputs, targeting a minimum mean Average Precision (mAP50) of 75% across all three violation classes.

4.6.2.3 Assumptions and Dependencies

In designing this project, the researcher makes the following assumptions related to the software, hardware, and deployment environment:

- The project follows the Agile methodology throughout development.
- The detection pipeline and the Django dashboard communicate through a REST-style API over the local network.
- The researcher writes the application in Python for the detection pipeline and Django backend, with HTML, CSS, and JavaScript for the dashboard front end.
- The system uses a relational database where all violation records and user information are saved.
- The Tapo C310 camera and the host computer remain connected to the same local network during operation, with the camera assigned a static IP address.
- Camera placement provides adequate fixed-angle coverage of the monitored pool zones with sufficient lighting for reliable image capture.
- The trained YOLOv8 model (best.pt) is available on the host machine prior to full-pipeline operation.
- The system must be easy to interact with and provide accurate violation information for effective rule enforcement decision-making.

4.6.2.4 Security Requirements

Given that the system stores patron incident imagery, data security and privacy are paramount. Several measures are implemented to ensure the security of stored data and prevent unauthorized access:

User Authentication and Role-Based Access Control: A secure login mechanism built on Django's authentication framework prevents unauthorized dashboard access. User access varies by role (administrator or staff), following the principle of least privilege.

Data Protection: All user credentials are hashed using Django's built-in password hashing. Violation records and screenshots are stored locally on the host computer and are not transmitted to external servers or cloud services.

Input Validation and Error Handling: The Django application validates all API inputs from the detection pipeline and all user inputs from the dashboard, protecting against SQL injection through Django's object-relational mapping layer, which uses parameterized queries inherently.

Access Control: Dashboard sessions are protected by login credentials and session management policies, ensuring users can only view and interact with data pertinent to their responsibilities.

4.7 System Design

The System Design phase translates the gathered requirements into a blueprint for constructing the VisionPool system, detailing how the system is structured and how its components interact. The design phase involves both high-level architectural design and detailed design of system components, ensuring the final product is robust, scalable, and maintainable.

4.7.1 Database Design

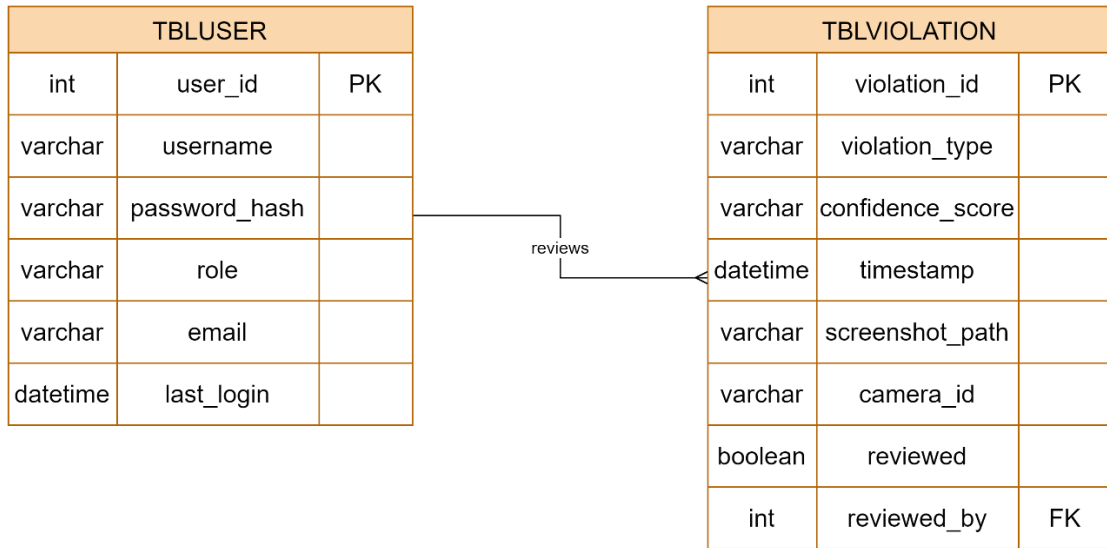


Figure 6. Database Design

The VisionPool database is designed using a relational schema to efficiently store and retrieve violation records and administrative user data. The primary table, `tblViolation`, records each detected violation incident with its type, confidence score, timestamp, and screenshot reference. The `tblUser` table manages dashboard access credentials and role assignments for authorized pool management personnel.

4.7.2 Entity Relationship Diagram

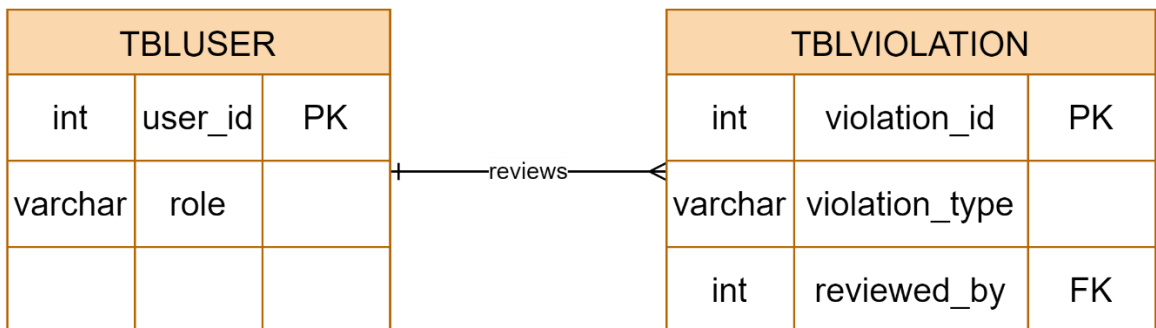


Figure 7. Entity Relationship Diagram

The Entity Relationship Diagram illustrates the data relationships within the VisionPool system. Each violation record is an independent entity created automatically by the detection pipeline. Authorized users access and review violation records through the dashboard. The relationship between tblUser and tblViolation is a one-to-many relationship, where a single authorized user may review multiple violation records.

4.7.3 Database Fields

A field is a data structure for a single piece of data. Fields are organized into records, which contain all the information within the table relevant to a specific entity. To keep the data consistent from one record to the next, the appropriate data type is assigned to each column. The primary key of each table uses an auto-incrementing integer, which automatically generates a unique number in each row. A primary key (PK) is a unique identifier for a given entity.

TABLE tblViolation				
Attribute Name	Data Type	Max Length	Key Type	Null
violation_id	INTEGER	11	Primary Key	No
violation_type	VARCHAR	100	—	No
confidence_score	FLOAT	—	—	No
timestamp	DATETIME	—	—	No
screenshot_path	VARCHAR	500	—	Yes
camera_id	VARCHAR	50	—	Yes
reviewed	BOOLEAN	—	—	No
reviewed_by (user_id)	INTEGER	11	Foreign Key	Yes

Table 8. TBLViolations

TABLE tblUser				
Attribute Name	Data Type	Max Length	Key Type	Null
user_id	INTEGER	11	Primary Key	No
username	VARCHAR	150	—	No
password_hash	VARCHAR	255	—	No
role	VARCHAR	50	—	No
email	VARCHAR	255	—	Yes
last_login	DATETIME	—	—	Yes

Table 9. TBLUsers

4.8 Tools and Technology

This section outlines the tools, platforms, and technologies used throughout the development and deployment of the system. The choice of tools was based on their compatibility with the project requirements, ease of use, availability of resources, and support for integration and scalability.

- Programming Language: Python 3.10+ was used for the detection pipeline and Django backend due to its extensive machine learning ecosystem and open-source nature.
- Object Detection Framework: Ultralytics YOLOv8s was used for real-time violation detection, fine-tuned on the custom pool environment dataset.
- Computer Vision Library: OpenCV 4.x was used for RTSP camera stream capture, frame extraction, bounding box rendering, and screenshot saving.
- Web Framework: Django 4.x served as the backend framework for the administrative dashboard, database object-relational mapping, and user authentication.

- Database: SQLite served as the primary relational database during development, with PostgreSQL identified for production deployment.
- Frontend Technologies: HTML, CSS, and JavaScript were utilized through Django templates to design a responsive and user-friendly dashboard interface.
- Dataset Management: Roboflow was used for image annotation, dataset augmentation, and export in YOLOv8 format.
- Model Training Environment: Google Colaboratory with GPU runtime was used for training the YOLOv8 model.
- Camera Hardware: TP-Link Tapo C310 IP camera, providing RTSP streaming at up to 2304x1296 resolution over the local network.
- Stream Verification: VLC Media Player was used to verify the RTSP stream during camera setup.
- Version Control: Git and GitHub were used for source code management and version tracking.
- Development Environment: Visual Studio Code was used as the primary integrated development environment.

These tools facilitated the efficient development, testing, and deployment of the system, ensuring seamless integration across the detection pipeline, database, and dashboard modules.

4.9 Evaluation and Testing

This section describes the strategies and procedures employed to evaluate the functionality, usability, and performance of the system. The goal is to ensure the system meets user requirements and operates reliably in real-world pool facility conditions.

- Model Performance Evaluation: The trained YOLOv8 detection model is evaluated on the held-out test dataset using precision, recall, F1 score, and mAP50 metrics. A minimum mAP50 of 75% across all three violation classes is set as the target performance threshold. Per-class precision and recall values are analyzed to identify detection weaknesses requiring additional training data or annotation refinement.
- Unit Testing: Individual system components, including the RTSP stream capture module, YOLOv8 inference function, screenshot capture mechanism, database

write operations, and dashboard view functions, are tested independently to verify that each behaves as expected.

- **Integration Testing:** The complete pipeline is tested end-to-end with the live Tapo C310 camera to verify proper data flow from video capture through violation detection, screenshot logging, and dashboard display. System latency from violation occurrence to dashboard display is measured against the three-second performance requirement.
- **User Acceptance Testing (UAT):** Pool management staff and administrators from the target facility test the system dashboard under realistic conditions and evaluate it using a structured evaluation form based on a five-point Likert scale, covering dashboard usability, violation alert clarity, screenshot quality, and overall system usefulness.
- **Performance Testing:** The system is evaluated for detection frame rate, response time, and CPU and memory utilization during continuous operation under typical workloads to verify stable performance at the camera's 15 frames per second stream rate.
- **Accuracy and Reliability:** Detection outputs are cross-checked against ground-truth annotations and human observer records during live testing sessions to validate that flagged violations correspond to actual rule violations and to quantify false positive and false negative rates.

Feedback from these evaluations is used to refine the detection model, adjust the confidence threshold, and improve dashboard usability, ensuring the system provides reliable support for pool rule compliance management.

REFERENCES

- [1] S. Few, *Information Dashboard Design: The Effective Visual Communication of Data*. Sebastopol, CA: O'Reilly Media, 2006.
- [2] N. D. Nath, A. H. Behzadan, and S. G. Paal, "Deep Learning for Site Safety: Real-Time Detection of Personal Protective Equipment," *Automation in Construction*, vol. 112, p. 103085, 2020.
- [3] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, *Dive into Deep Learning*. Cambridge, UK: Cambridge University Press, 2023.
- [4] Y. LeCun, Y. Bengio, and G. Hinton, "Deep Learning," *Nature*, vol. 521, no. 7553, pp. 436–444, 2015.
- [5] Django Software Foundation, "Django Documentation," 2024. [Online]. Available: <https://docs.djangoproject.com>
- [6] M. Loey, G. Manogaran, M. H. N. Taha, and N. E. M. Khalifa, "A Hybrid Deep Transfer Learning Model with Machine Learning Methods for Face Mask Detection in the Era of the COVID-19 Pandemic," *Measurement*, vol. 167, p. 108288, 2021. doi: 10.1016/j.measurement.2020.108288
- [7] Q. He, Z. Mei, H. Zhang, and X. Xu, "Automatic Real-Time Detection of Infant Drowning Using YOLOv5 and Faster R-CNN Models Based on Video Surveillance," *Journal of Social Computing*, vol. 4, no. 1, pp. 62–73, 2023. doi: 10.23919/JSC.2023.0006
- [8] M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, "The PASCAL Visual Object Classes (VOC) Challenge," *International Journal of Computer Vision*, vol. 88, no. 2, pp. 303–338, 2010.
- [9] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

- [10] C.-J. Lin and J.-Y. Jhang, "Intelligent Traffic-Monitoring System Based on YOLO and Convolutional Fuzzy Neural Networks," *IEEE Access*, vol. 10, pp. 14120–14133, 2022. doi: 10.1109/ACCESS.2022.3147866
- [11] Roboflow, "Roboflow: Build and Deploy Computer Vision Applications," 2024. [Online]. Available: <https://roboflow.com>
- [12] H. Schulzrinne, A. Rao, and R. Lanphier, "Real Time Streaming Protocol (RTSP)," RFC 2326, Internet Engineering Task Force, Apr. 1998.
- [13] G. Jocher, A. Chaurasia, and J. Qiu, "Ultralytics YOLOv8," Ultralytics, 2023. [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [14] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," *arXiv preprint arXiv:1804.02767*, 2018.
- [15] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," *arXiv preprint arXiv:2004.10934*, 2020.
- [16] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 779–788.
- [17] N. S. Pun, S. K. Sonbhadra, S. Agarwal, and G. Rai, "Monitoring COVID-19 Social Distancing with Person Detection and Tracking via Fine-Tuned YOLO v3 and Deepsort Techniques," *arXiv preprint arXiv:2005.01385*, 2020.
- [18] R. Yang, K. Wang, and L. Yang, "An Improved YOLOv5 Algorithm for Drowning Detection in the Indoor Swimming Pool," *Applied Sciences*, vol. 14, no. 1, p. 200, 2024. doi: 10.3390/app14010200
- [19] N. D. Nath, A. H. Behzadan, and S. G. Paal, "Deep Learning Detection of Personal Protective Equipment to Maintain Safety Compliance on Construction Sites," in *Proc. Construction Research Congress 2020*, Tempe, AZ, USA, 2020, pp. 181–190.

- [20] K. Simonyan and A. Zisserman, "Two-Stream Convolutional Networks for Action Recognition in Videos," in *Advances in Neural Information Processing Systems* (NeurIPS), 2014, pp. 568–576.
- [21] Z. Cao, G. Hidalgo, T. Simon, S.-E. Wei, and Y. Sheikh, "OpenPose: Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 43, no. 1, pp. 172–186, 2021. doi: 10.1109/TPAMI.2019.2929257
- [22] "Research on an Elderly Indoor Fall Detection System Based on IoT and Computer Vision Technology: Integration and Performance Analysis of YOLOv5 and OpenPose," *Highlights in Science, Engineering and Technology*, vol. 119, 2024.
- [23] C. Direkoglu, "Abnormal Crowd Behavior Detection Using Motion Information Images and Convolutional Neural Networks," *IEEE Access*, vol. 8, pp. 80408–80416, 2020. doi: 10.1109/ACCESS.2020.2990355
- [24] Z. Liu, P. Luo, S. Qiu, X. Wang, and X. Tang, "DeepFashion: Powering Robust Clothes Recognition and Retrieval with Rich Annotations," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, Las Vegas, NV, USA, 2016, pp. 1096–1104.
- [25] A. Aboah, B. Wang, U. Bagci, and Y. Adu-Gyamfi, "Real-Time Multi-Class Helmet Violation Detection Using Few-Shot Data Sampling Technique and YOLOv8," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2023, pp. 5350–5358.
- [26] M. Loey, G. Manogaran, M. H. N. Taha, and N. E. M. Khalifa, "Fighting Against COVID-19: A Novel Deep Learning Model Based on YOLO-v2 with ResNet-50 for Medical Face Mask Detection," *Sustainable Cities and Society*, vol. 65, p. 102600, 2021.

[27] K. Bhambani, T. Jain, and K. A. Sultanpure, "Real-Time Face Mask and Social Distancing Violation Detection System Using YOLO," in 2020 IEEE Bangalore Humanitarian Technology Conference (B-HTC), Vijayapura, India, 2020, pp. 1–6.